

T04. Introducción a MySQL.

Características MySQL.....	1
Comprobar el servicio.....	2
Instalación.....	2
Comprobar el servicio.....	2
Entrar/salir a MySQL.....	3
Comprobar usuarios MySQL.....	3
Comandos útiles.....	4
Usuarios.....	4
Scripts.....	5
Lenguaje DDL.....	6
Creación de bases de datos.....	6
Modificar / Eliminar base de datos.....	7
Creación de tablas.....	7
Eliminar tabla.....	8
Insertar, consultar, modificar y borrar registros de una tabla.....	9
Insertar.....	9
Consultar.....	9
Modificar.....	9
Borrar.....	10
Modificar estructura de una tabla.....	10
Copia de seguridad.....	11
mysqldump.	11
Restaurar una copia de seguridad.....	12
Tipos de datos.....	13
Tipos numéricos.....	13
Números enteros.....	13
Números en coma flotante.....	14
Números en coma fija.....	14
Fechas y Tiempo.....	15
Cadenas de caracteres.....	15
Datos binarios.....	15
Enumeraciones.	16
Valores fuera de rango.....	16
Ejercicios.....	17

Características MySQL.

- Es un Gestor de Bases de Datos.
- Gestiona base de datos relacionales. Para usar y gestionar las bases de datos utiliza el lenguaje estándar SQL.
- Es open source: el código está disponible y tiene licencia GNU para uso no comercial.
- Rápido, seguro y fácil de usar. Razón por la que es una de las más usadas en Internet.

MySQL dispone de un interfaz gráfico basado en página web lo que permite acceder desde cualquier equipo: PhpMyAdmin. Las páginas web se sirven con el servidor Apache. Además, también está disponible el entorno gráfico de workbench.

El servicio MySQL se ofrece desde el puerto 3306.

Comprobar el servicio.

Instalación.

```
#apt update  
#apt install mysql-server
```

Comprobar el servicio.

```
#systemctl start mysql  
#systemctl stop mysql  
#systemctl restart mysql  
#systemctl status mysql
```

O como alternativa:

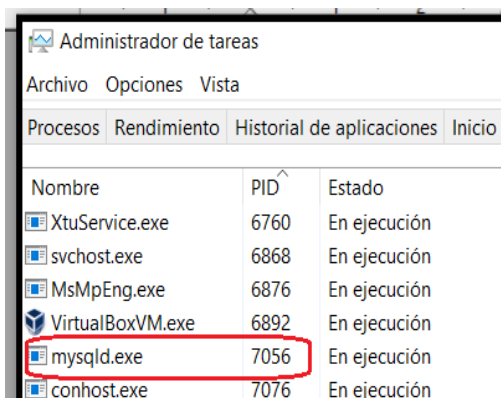
```
#!/etc/init.d/mysql stop  
#!/etc/init.d/mysql start  
#!/etc/init.d/mysql restart  
#!/etc/init.d/mysql status  
#!/etc/init.d/mysql reload  
#!/etc/init.d/mysql force-reload
```

Como hemos comentado el servidor de MySQL escucha (espera recibir las peticiones de los clientes) por el puerto 3306, por lo que si analizamos los puertos podemos ver si realmente está en funcionamiento. El comando netstat permite hacer un escaneo de puertos y según las opciones que se indiquen nos responderá.

En Windows:

```
c:\> netstat -aon | findstr 3306
```

El listado mostrado nos indica el PID de cada proceso afectado. Para saber cual es el proceso: Ctrol +Alt + Spr /Administrador de tareas / Pestaña detalles. Ordenar por PID y buscar el número indicado.



En Linux:

#netstat -h	Mostrar ayuda de opciones
#netstat -ltun	Muestra los números de procesos
#netstat -ltu	Muestra los procesos asociados

El significado de las opciones es:

- l Listado
- t Puertos TCP
- u Puertos UDP
- n Número de puerto

Si netstat no estuviese disponible se instala con:

```
#apt install net-tools
```

Entrar/salir a MySQL.

Entrada:

```
$mysql [-u usuario -p] [-h ipservidormysql]
```

Desde el sistema operativo podemos pedir ayuda para entrar en mysql:

```
$mysql --help
```

Una alternativa para entrar es via telnet:

```
$telnet ipservidormysql 3306
```

Una vez dentro podemos usar para pedir ayuda:

```
mysql>help
```

Para Salir:

```
mysql>quit
```

```
mysql>exit
```

Otra alternativa es Ctrol+D

Comprobar usuarios MySQL.

Dentro de mysql hay una base de datos llamada “mysql” que contiene información en general. Para utilizar una base de datos primero hay que seleccionarla. Para ello se utiliza el comando “USE”:

```
mysql>USE mysql;
```

Para ver el contenido de una tabla hay que especificar que atributos queremos ver y de que tabla de las que estén definidas en la base de datos seleccionada. Esto se hace con el comando “SELECT”:

```
mysql>SELECT user, host, plugin FROM user;
```

user: el primer user es el nombre de un atributo de la tabla user (segundo user).

Ese comando nos mostraría los usuarios actuales de mysql, desde donde pueden acceder y si la contraseña que utilizan es la del sistema o es propia de mysql.

Comandos útiles.

Podemos distinguir entre comandos y funciones. Un comando es una instrucciones con la que se le indica al SGBD que realice una acción sin esperar ningún dato como resultado de la ejecución de la misma, aunque si pueden mostrar un listado. Ejemplo: crear una base de datos o una tabla, borrar un usuario,.. Sin embargo las funciones son instrucciones que realizan un cálculo y nos devuelve su resultado, como puede ser: ¿quién es el usuario actual?, ¿qué fecha es hoy?..

Algunos comandos de interés:

SHOW DATABASES;	Muestra las bases de datos disponibles.
SHOW TABLES;	Muestra las tablas de una base de datos.
CREATE DATABASE nombredb;	Crea la base de datos indicada.
DROP DATABASE nombredb;	Borra la base de datos indicada.
USE nombredb;	Selecciona la base de datos indicada.
DESCRIBE nombretabla;	Muestra la estructura de una tabla.
SHOW CREATE DATABASE;	Muestra la instrucción de crear un base de datos.
SHOW CREATE TABLE;	Muestra la instrucción de crear una tabla.

Algunas funciones de interés:

SELECT version();	Muestra la versión del SO y de MySQL.
SELECT current_date();	Muestra la fecha de hoy.
SELECT user();	Muestra el usuario actual.
SELECT now();	Muestra fecha y hora.

Usuarios.

Los usuarios de la base de datos han de crearse con una contraseña y definirle los permisos con los que acceden a MySQL. Cada nombre de usuario está formado por dos partes:

- El nombre del usuario en sí.
- El ordenador desde el que está autorizado a acceder: puede tener los siguientes valores:
 - localhost: El ordenador donde está instalado el servidor MySQL.
 - N° de IP o dominio: Se especificará desde que IP o nombre de dominio se está autorizado.
 - %: Es un comodín que indica que se puede acceder desde cualquier ordenador.

Las operaciones más comunes sobre usuarios son las siguientes (Por simplificar la sintaxis lo expresaremos todo como si solo se puede acceder desde el ordenador donde está instalado el servidor: 'localhost'):

1. Creación/eliminación de usuarios.
mysql>CREATE USER nombre@localhost IDENTIFIED BY 'contraseña';
mysql>DROP USER nombre@localhost;

2. Asignación/eliminación de privilegios

mysql>GRANT [permiso] ON [basedato].[tabla] TO usuario@localhost;

Donde permiso puede ser:

- ALL PRIVILEGES: Permite acceso a todas las bases de datos del sistema.
- CREATE: Permite crear bases de datos y tablas.
- DROP: Permite eliminar bases de datos y tablas.
- DELETE: Permite eliminar registros de tablas.
- INSERT: Permite insertar registros de tablas.
- SELECT: Permite leer registros de tablas.
- UPDATE: Actualiza registros seleccionados de tablas.
- GRANT OPTION: Permite asignar privilegios a terceros (otros usuarios). Solo afecta los permisos que se le concedan con GRAN OPTION.

Para eliminar los privilegios:

mysql>REVOKE [permiso] ON [basedatos].[tabla] FROM usuario@localhost;

Para ver los permisos de un usuario determinado:

mysql>SHOW GRANTS FOR usuario;

3. Después de asignar o eliminar los privilegios es necesario refrescar el sistema. Esto se hace con el comando:

mysql>FLUSH PRIVILEGES;

4. Cambiar contraseña. Es una tarea que se ha de hacer con regularidad para mejorar la seguridad de los accesos.

mysql>ALTER USER nombre_usuario@localhost IDENTIFIED BY “contraseña_nueva”;

Scripts.

Conocer los comandos y las instrucciones que proporciona un SGBD ofrece una visión extra que posibilita automatizar tareas rutinarias y permite solucionar problemas que no se pueden solucionar desde el entorno gráfico.

Los comandos se pueden poner en un fichero de texto plano y se ejecutarán uno detrás de otro, por lo que el orden en el aparezcan es importante (caso de dependencias de claves foráneas, por ejemplo). Esos scripts pueden ser llamados desde el SO o desde el propio entorno de comandos del servidor MySQL.

- Desde el SO:

\$mysql -u usuario -p < nombre_script > resultado

Donde “resultado” será un fichero donde se almacenará el resultado de la ejecución del script.

- Desde interprete de comandos de mysql:

mysql> source **nombre_script**;

La extensión del script ha de ser .sql

El script se puede documentar utilizando el carácter “#” para indicar que una línea es un comentario.

Otro aspecto importante de gestionar es el envío de un script a otro ordenador. Para ello utilizaremos el comando **scp**, que realiza la transferencia encriptada mediante ssh. Es necesario tener permiso de escritura en el destino.

Sintaxis:

```
> scp [opciones] origen destino
```

Donde opciones:

- P Port permite especificar una entrada diferente al servidor (el puerto predeterminado para el puerto TCP para el comando es 22).
- q Ejecutará la operación en modo silencioso (quiet), lo que significa que solo se mostrarán los errores críticos.
- r Es para copia recursiva, que incluirá todos los subdirectorios.
- 4 ó -6 Se pueden usar si quieres elegir la versión de protocolo empleada, ya sea IPv4 o Ipv6. Por defecto IPv4.
- p Conservará los tiempos de modificación iniciales y los atributos del archivo.
- u Borrará el archivo fuente después de que se complete la transferencia.
- c Permitirá la compresión de los datos mientras se lleva a cabo la operación de transferencia.

Origen o destino: [usuario@ordenador](#):/directorio/nombre_fichero. Si estamos en el directorio o carpeta donde esta el fichero a transferir no hace falta especificar ni el usuario ni el ordenador.

Ejemplos: >scp veterinariodb.sql [joseant@146.59.232.252](#):/home/joseant/veterinariodb.sql
 >scp [joseant@146.59.232.252](#):/home/joseant/datos.sql datos.sql

En el primer caso transferimos a un ordenador remoto el archivo “veterinariodb.sql” y en el segundo caso transferimos desde el ordenador remoto al local “datos.sql”.

Lenguaje DDL.

El lenguaje DDL (Data Definition Language) o Lenguaje de Definición de Datos está formado por los comandos que nos permiten definir los datos en el SGBD, estos son:

- CREATE: permite crear objetos tales como bases de datos o esquemas, tablas e índices.
- DROP: permite eliminar un objeto existente.
- ALTER: permite modificar la definición de un objeto.
- SHOW: permite consultar los objetos de la base de datos.

Otras sentencias o comandos de utilidad:

- USE: con ella especificamos la base de datos con la que se va a trabajar.
- DESCRIBE: permite mostrar información sobre la estructura de una tabla.

Creación de bases de datos.

Sintaxis:

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] nombre_bd  
[especificación_create[, especificación_create]..];
```

Donde especificación_create:

```
[DEFAULT] CHARACTER SET juego_caracteres  
| [DEFAULT] COLLATE nombre_colación
```

Los términos DATABASE y SCHEMA son sinónimos, solo se mantienen por compatibilidad con otros SGBD.

La base de datos se creará si no existe. Pero, ¿qué juego de caracteres utilizará? y ¿cómo se van a ordenar los símbolos de esos caracteres?, ¿es lo mismo “á” que “a” o “A”? Eso es lo que se define con el juego de caracteres y la colación. Se puede ver un listado completo de todas las disponibles:

```
mysql> SHOW CHARACTER SET;  
mysql> SHOW COLLATION;
```

El juego de caracteres si no se especifica se toma por defecto **latin1**.

La colación puede ser:

- case-sensitive (_cs): las mayúsculas y las minúsculas las trata distintas.
- case-insensitive (_ci): las mayúsculas y las minúsculas las trata igual.
- binary (_bin): dos caracteres son iguales si los valores de su representación numérica son iguales.

Utilizaremos el juego de caracteres: latin1 y la colación: latin1_spanish_ci

Para mostrar las bases de datos activas en el servidor se utiliza el comando:

```
mysql> SHOW DATABASES;
```

Por defecto aparecen las bases de datos: mysql y information_schema, estas contienen metadatos de los usuarios y permisos del SGBD.

Para usar una base de datos primero se pone en uso con el comando:

```
mysql> USE nombredb;
```

Se puede ver que comando se ha utilizado para crear una determinada base de datos:

```
mysql> SHOW CREATE DATABASE nombre_bd;
```

Modificar / Eliminar base de datos.

Solo se permite modificar el juego de caracteres o la colación. La sintaxis es:

```
mysql> ALTER DATABASE nombre_bd COLLATE nombre_colación;  
mysql> ALTER DATABASE nombre_bd CHARACTER SET nombre_caracteres;
```

Para eliminar una base de datos:

```
mysql> DROP {DATABASE | SCHEMA} [IF EXISTS] nombre_bd;
```

Creación de tablas.

Sintaxis.

```
CREATE TABLE [IF NOT EXISTS] nombre_tabla  
    (descripcion_campos)  
    [opciones];
```

Descripcion_campos, cada campo requiere una estructura como la siguiente y van separados por comas:

```
nombre_campo tipo_dato [NOT NULL | NULL] [DEFAULT valor] [AUTO_INCREMENT]  
[UNIQUE [KEY] | [PRIMARY] KEY]
```

Si hay claves compuestas, por estilo se recomienda ponerlo detrás de la definición de todos los campos:

```
[CONSTRAINT [nombre_constraint]] PRIMARY KEY (nombre_campo1,..)
```

Si hay claves foráneas, por estilo se recomienda ponerlo detrás de la definición de los campo y de la clave primaria si fuese compuesta:

```
[CONSTRAINT [nombre_constraint]] FOREIGN KEY (nombre_campo1,..)  
REFERENCES nombre_tabla (nombre_campo1,..)  
[ON DELETE opcion_referencia]  
[ON UPDATE opcion_referencia]
```

Opcion_referencia:

```
RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT
```

RESTRICT: usado por defecto por MySQL. Impide que se puedan actualizar o eliminar las filas de los valores referenciados por las claves ajenas.

CASCADE: Permite actualizar o eliminar las filas que tienen valores referenciados por las claves ajenas.

SET NULL: Asigna NULL a las filas que tienen valores referenciados por claves ajenas.

NO ACTION: en MySQL es equivalente a RESTRICT.

SET DEFAULT: no es posible usar esta opción si se utiliza el motor de de almacenamiento InnoDB.

Opciones, al definir una tabla se pueden establecer las siguientes opciones:

```
AUTO_INCREMENT [=] valor  
| [DEFAULT] CHARACTER SET [=] character_name  
| [DEFAULT] COLLATE [=] collation_name  
| ENGINE [=] engine_name
```

AUTO_INCREMENT: especifica el valor inicial que se usará en un campo AUTO_INCREMENT.

CHARACTER SET: especifica el juego de caracteres que usará la tabla.

COLLATE: especifica el tipo de cotejamiento que vamos a usar en la tabla.

ENGINE: especifica el tipo de almacenamiento que tendrá la tabla. Los más habituales en MySQL son InnoDB y MyISAM. Por defecto es InnoDB.

Eliminar tabla.

```
DROP TABLE [IF EXISTS] nombre_tabla;
```

Se eliminará la tabla especificada con “nombre_tabla”.

Es conveniente proteger la sentencia con la opción: “IF EXISTS”, sobre todo en procesos automáticos y/o en scripts, de otra forma daría error si no existe la tabla.

Insertar, consultar, modificar y borrar registros de una tabla.

Insertar.

El comando para introducir datos en una tabla es INSERT INTO que pertenece al lenguaje de manipulación de datos, DML. Se verá con detalle más adelante. Pero, para poder introducir algunos datos en las tablas que creemos y hacer pruebas, vamos a ver una introducción:

```
INSERT INTO nombre_tabla [(campo1, campo2,...)] VALUES (valor1, valor2,..);
```

Donde la lista de campos: campo1, campo2,.. son campos de la tabla y la lista de valores: valor1, valor2,.. la lista de valores que se le asignaran a esos campos por orden de aparición en el nuevo registro.

Si la lista de campos no aparece, entonces en la lista de valores han de aparecer todos los campos de la tabla y en el mismo orden en el que se definieron.

Consultar.

De la misma forma el comando para hacer las consultas es: SELECT, que merece una atención especial por la variedad de opciones que ofrece. Para adelantar de forma simplificada su funcionamiento:

```
SELECT {campo1, campo2,.. | *} FROM nombre_tabla;
```

Si se especifica la lista de campos, mostrará solo esos campos y en el orden indicado. Con el “*” muestra todos los campos de la tabla.

Modificar.

Si necesitamos modificar algún dato de la tabla se utiliza el comando UPDATE. Cuya sintaxis abreviada es:

```
UPDATE nombre_tabla SET nombre_atributo = valor [, nombre_atributo = valor]..  
[WHERE condición]
```

En una instrucción se pueden cambiar más de un atributo a la vez, habría que separar por coma cada pareja de “atributo=valor”.

Si no se especifica WHERE, se cambiarán todos los registros de la tabla.

Ejemplos:

```
UPDATE mascota SET raza="podenco" WHERE codigo=12;
```

Cambiará el contenido de “raza” a “podenco” del registro cuyo “codigo” es “12” en la tabla “mascota”.

Si queremos que el cambio afecte a un conjunto de registros podemos especificar la condición IN (lista): WHERE codigo IN (2,4,5,10); cambiaría “raza” a esos cuatro registros.

Nota: Si se intenta de actualizar o borrar registros y MySQL tiene activado el “modo seguro de actualización”, solo nos dejará hacerlo si en la condición trabajamos con la clave principal. En caso contrario aparecerá el error: 1175.

El modo seguro de actualización se gestiona a través de la variable del sistema:

SQL_SAFE_UPDATES.

Para activarla: SET SQL_SAFE_UPDATES=1;

Para desactivarla: SET SQL_SAFE_UPDATES=0;

Borrar.

La instrucción DELETE nos permite borrar registros de una tabla. Se debe de usar siempre con la cláusula “WHERE condición”, pues si no se hace así se eliminarán todos los registros de la tabla.

La sintaxis es como sigue:

```
DELETE FROM nombre_tabla [WHERE condición];
```

Ejemplos:

```
mysql> DELETE FROM mascota;
```

```
mysql> DELETE FROM mascota WHERE nombre = “Misifú”;
```

En el primer caso borraría todos los registros de la tabla “mascota”. Y en el segundo solo borraría el registro o registros que tengan en el atributo nombre el valor “Misifú”.

Modificar estructura de una tabla.

En muchas ocasiones hace falta modificar los atributos de una tabla: añadir o eliminar algún campo, cambiar el tipo de dato o su longitud, establecer o eliminar alguna clave foránea,.. Si la tabla no tiene datos se podría eliminar y volver a crearla, pero si tiene datos se utiliza la sentencia ALTER TABLE y no se pierde la información.

Sintaxis:

```
ALTER TABLE nombre_tabla
```

```
[especificación [, especificación]..]
```

Especificaciones (usar solo una),:

- ADD [COLUMN] nombre_columna definicion_columna [FIRST | AFTER nombre_columna]
- ADD {INDEX|KEY} [nombre_indice] [tipo_indice] (columna_indice)
- ADD [CONSTRAINT [simbolo]] PRIMARY KEY [tipo_indice] (columna_indice)

- ADD [CONSTRAINT [simbolo]] FOREIGN KEY [nombre_indice] (**columna_indice**,...) REFERENCES nombre_tabla (nombre_campo1,...)
[ON DELETE opcion_referencia]
[ON UPDATE opcion_referencia]
- ALTER [COLUMN] nombre_columna {SET DEFAULT literal | DROP DEFAULT}
- CHANGE [COLUMN] nombre_columna_anterior nombre_columna_nuevo definicion_columna [FIRST | AFTER nombre_columna]
- [DEFAULT] CHARACTER SET [=] nombre_caracteres [COLLATE [=] nombre_collate]
- CONVERT TO CHARACTER SET nombre_caracteres [COLLATE nombre_collate]
- DROP [COLUMN] nombre_columna
- DROP {INDEX | KEY} nombre_indice
- DROP PRIMARY KEY
- DROP FOREIGN KEY fk_simbolo
- MODIFY [COLUMN] nombre_columna definicion_columna [FIRST | AFTER nombre_columna]
- ORDER BY nombre_columna [, nombre_columna]..
- RENAME {INDEX | KEY} nombre_indice_anterior nombre_indice_nuevo
- RENAME [TO | AS] nuevo_nombre_tabla

Donde **columna_indice** será:

nombre_columna [(longitud)] [ASC | DESC]

Tipo_indice:

USING {BTREE | HASH}

Para más información utilizar la información oficial de MySQL.

Copia de seguridad.

Es importante hacer una copia de seguridad de forma periódica de nuestra/s base/s de datos y así poder recuperarla en caso de alguna incidencia.

Hay dos posibilidades: copia lógica y copia física.

En la **copia lógica** se exporta: bases de datos, tablas y contenidos; todo ello a un fichero de texto plano con las instrucciones SQL: CREATE e INSERT (un script). Esto permite poder editarlo antes de recuperarlo si fuese necesario. Se realiza con el comando: mysqldump

Por otra parte, con la **copia física** solo se almacenan los datos. Hay dos utilidades para hacerlo según tengamos un sistema de almacenamiento InnoDB o MyISAM. Estas utilidades son: `mysqlbackup` para InnoDB y `mysqlhotcopy` para MyISAM. Sólo están disponibles en MySQL Enterprise Edition.

La copia física es más rápida que la copia lógica.

mysqldump.

Se puede usar de tres formas.

- Exportar una o varias tablas de una base de datos.

mysqldump [opciones] nombre_db [tabla1 tabla2 ..] > fichero_backup.sql

Exporta solo de una base de datos. Si no se especifican las tablas se copia la base de datos completa. La copia de seguridad se almacena en fichero_backup.sql

Con este formato no se incluyen las ordenes: CREATE DATABASE, pero sí las de CREATE TABLE.

- Exportar una o varias bases de datos completas.

mysqldump [opciones] --databases nombre_db .. > fichero_backup.sql

Se exportarían todas las tablas de la base/s de datos especificada/s. No se pueden especificar las tablas individuales. La copia de seguridad se almacena en fichero_backup.sql

Si se incluyen las ordenes: CREATE DATABASE y CREATE TABLE.

- Exportar todas las bases de datos.

mysqldump [opciones] --all-databases > fichero_backup.sql

Se exportarían todas las bases de datos completas que estuviesen en el servidor. La copia de seguridad se almacena en fichero_backup.sql

Algunas opciones que se pueden especificar:

-u	Usuario
-p	Contraseña
-h	Host

Ejemplos:

- `mysqldump -u root -p bibliotecadb > backup.sql`

Se hace una copia de seguridad de todas las tablas de la base de datos bibliotecadb y se almacena en el fichero backup.sql. No se incluye la orden CREATE DATABASE, pero si las ordenes: CREATE TABLE e INSERT que sean necesarias.

- `mysqldump -u root -p --databases wordpress mediawiki > backup.sql`

Se hace la copia de seguridad de las bases de datos: wordpress y mediawiki. En este caso si se incluyen las ordenes CREATE DATABASE, CREATE TABLE e INSERT.

- `mysqldump -u root -p --all-databases > backup.sql`

Se copiarían todas las bases de datos del servidor MySQL incluyendo las ordenes de crear las bases de datos, las tablas e insertar los datos.

Restaurar una copia de seguridad.

Para recuperar una copia de seguridad se procede igual que cuando se ejecuta un script. Solo hay que tener en cuenta si la copia se hizo con `--databases` o con `--all-databases` en cuyos casos están incluidas la creaciones de las bases de datos. En el otro caso habría que editar el fichero de backup e incluir la orden de crear la base de datos o ponerla en uso (USE).

Desde el sistema operativo:

```
$mysql -u root -p < backup.sql
```

Desde el entorno mysql:

```
mysql>source backup.sql;
```

Tipos de datos.

Es importante especificar el tipo de dato de cada campo o atributo lo más ajustado a la realidad de los contenidos que albergará. Esto repercutirá en eficacia tanto en espacio de almacenamiento como en velocidad de proceso.

Un tipo de dato concreto, además de indicar que valores puede tener, indica las operaciones que se puede realizar sobre él.

Básicamente los tipos de datos son: numéricos, caracteres, binarios, fecha-hora y booleanos. Pero dentro de estos tipos genéricos hay gran variedad de ellos para atender las distintas necesidades de uso. Ej: aún siendo numéricos, no es lo mismo almacenar una calificación que puede ser entre 0 y 10, que el presupuesto de una empresa que puede llegar a cientos de miles incluso millones.

Tipos numéricos.

Números enteros.

Tipo	Tamaño en bytes	Mínimo valor	Máximo valor.
TINYINT	1	-128	127
TINYINT UNSIGNED	1	0	255
SMALLINT	2	-32768	32767
SMALLINT UNSIGNED	2	0	65535
MEDIUMINT	3	-8388608	8388607
MEDIUMINT UNSIGNED	3	0	16777215
INT	4	-2147483648	2147483647
INT UNSIGNED	4	0	4294967295
INTEGER	4	-2147483648	2147483647
INTEGER UNSIGNED	4	0	4294967295
BIGINT	8	-0,922337E19	0,922337E19
BIGINT UNSIGNED	8	0	0,184467E20

ZEROFILL

Todos los tipos numéricos admiten el atributo ZEROFILL, esto incluye automáticamente el atributo UNSIGNED quedando: UNSIGNED ZEROFILL.

Se puede especificar detrás de INT y entre paréntesis un número: INT(N). Esto solo tiene sentido si se utiliza el atributo ZEROFILL pues para mostrar el contenido del campo mostrará como mínimo hasta N dígitos y si el valor tiene menos lo completa con ceros. Solo afecta a la forma de mostrar los datos.

Ej: supongamos un campo definido como INT(5) UNSIGNED ZEROFILL y que contiene el valor 23, por pantalla se mostrará: 00023. Si el valor fuese 120057, se mostrará 120057. Y siempre se almacenará en 4 bytes.

Otros tipos numéricos enteros:

BIT(M): M será un valor entre 1 y 64. Indica el número de bits que vamos a utilizar para ese campo. Si se omite el valor de M se utiliza un bit por defecto.

Observaciones. Podrá almacenar un entero positivo entre 0 y 2^M-1 . Ejemplo: un atributo definido como BIT(4), puede contener números entre 0 y 15.

El contenido de un atributo BIT, no se muestra directamente en la pantalla con SELECT, pero si tiene almacenado los valores que se hayan asignado. Para mostrarlo habría que pasarlo a un tipo numérico, simplemente sumando 0 se mostraría.

BOOL, BOOLEAN: Es equivalente a TINYINT(1). Si el valor es 0 se considera FALSE, en caso contrario TRUE.

SERIAL: Es un alias para BIGINT UNSIGNED NOT NULL AUTO_INCREMENT UNIQUE.

Números en coma flotante.

Se utiliza para representar números con decimales en notación científica: mantisa y exponente.

Con notación científica tenemos los siguientes tipos:

Tipo	Tamaño en bytes	Mínimo valor	Máximo valor.
FLOAT	4		
FLOAT(m,d)	4	-3,402823466E+38	-1,175494351E-38
FLOAT(m,d) UNSIGNED	4	1,175494351E-38	3,402823466E+38
DOUBLE	8		
DOUBLE (m,d)	8	-1,797693..E+308	-2,225073..E-308
DOUBLE(m,d) UNSIGNED	8	2,225073..E-308	1,797693..E+308

m: Indica el número de dígitos en total que se va a mostrar.

d: Indica el número de decimales que se tendrá en cuenta.

Ejemplo: Un campo declarado FLOAT(7,3) podría representar entre: -9999,999 y 9999,999.

Números en coma fija.

<u>Tipo</u>	<u>Tamaño en bytes.</u>
DECIMAL	
DECIMAL(m,d)	M + 2 bytes si d > 0; M + 1 bytes si d = 0
DECIMAL(m,d) UNSIGNED	M + 1 bytes si d > 0
NUMERIC	
NUMERIC(m,d)	M + 2 bytes si d > 0; M + 1 bytes si d = 0
NUMERIC(m,d) UNSIGNED	M + 1 bytes si d > 0

m: Indica el número de dígitos en total, entre 1 y 65.

d: Indica el número de cifras decimales, entre 0 y 30.

NUMERIC y DECIMAL son equivalentes. Este tipo de dato es apropiado para trabajar con cantidades económicas en los que no se haga redondeo.

Fechas y Tiempo.

<u>Tipo</u>	<u>Tamaño</u>	<u>Descripción</u>	<u>Mínimo</u>	<u>Máximo</u>
DATE	3	YYYY-MM-DD	1000-01-01	9999-12-31
DATETIME	8	YYYY-MM-DD HH:MM:SS	1000-01-01 00:00:00	9999-12-31 23:59:59
TIMESTAMP	4	YYYY-MM-DD HH:MM:SS	1970-01-01 00:00:00	2038-01-09 03:14:07
TIME	3	HH:MM:SS	-838:59:59	838:59:59
YEAR	[(2 4)]	YY o YYYY	YY: 70 (1970) YYYY: 1901	YY: 69 (2069) YYYY: 2155

Al definir el tipo de dato YEAR lo almacenará en 4 dígitos. Cuando se le asigne un valor si tiene dos cifras priorizará a YY, es decir, entre 1970 y 2069. Si no considerará que el rango del YYYY. Un valor fuera del rango YYYY (1901-2155) provocará un error.

Ej: si queremos almacenar el año 1935, se ha de usar el: 1935, pues si se usa 35 se considerará 2035.

Cadenas de caracteres.

<u>Tipo</u>	<u>Descripción</u>
CHAR(m)	m: ha de estar entre 1 y 255. Siempre ocupa m bytes. Rellena a blancos por la derecha hasta el tamaño especificado. Si no se especifica el tamaño es 1.
VARCHAR(m)	m: ha de estar entre 1 y 255. Almacenará cadenas de longitud variable hasta un máximo indicado por m. Se ha de especificar m. Cada campo ocupa entre 2 y m+1 bytes, según la longitud del contenido.
TEXT	Campo carácter de longitud máxima 65535 bytes. Ocuparía la longitud del contenido más 2 bytes.
MEDIUMTEXT	Campo carácter de longitud máxima 16 MB. Ocuparía la longitud del contenido más 3 bytes.
LONGTEXT	Campo carácter de longitud máxima 4GB. Ocuparía la longitud del contenido más 4 bytes.

Los tipos TEXT se utilizan para almacenar gran cantidad de texto y no soportan la cláusula DEFAULT. Los valores de TEXT se tratan como cadenas no binarias (cadenas de caracteres). Tienen un conjunto de caracteres y los valores se ordenan y comparan según la clasificación del conjunto de caracteres.

Datos binarios.

Estos tipos permiten almacenar gran cantidad de información en un campo. Esa información puede ser un fichero pdf, una imagen, audio, ..

No permiten la cláusula DEFAULT.

BLOB: Binary large Object.

Los valores BLOB se tratan como cadenas binarias (cadenas de bytes). No tienen juego de caracteres y la clasificación y la comparación se basan en los valores numéricos de los bytes que contengan cada campo.

<u>Tipo</u>	<u>Descripción.</u>
-------------	---------------------

BLOB	Campo de longitud máxima 65535 bytes. Ocuparía la longitud del contenido más 2 bytes.
------	---

MEDIUMBLOB	Campo de longitud máxima 16 MB. Ocuparía la longitud del contenido más 3 bytes.
------------	---

LOB	Campo de longitud máxima 4GB. Ocuparía la longitud del contenido más 4 bytes.
-----	---

Enumeraciones.

Una enumeración es lista de valores, de tal manera que aquel campo que se defina como una enumeración solo podrá contener como posible valor alguno de los que aparece en la lista o NULL.

<u>Tipo</u>	<u>Descripción.</u>
-------------	---------------------

ENUM('valor1', 'valor2', 'valor3',...)	Puede haber hasta 65535 valores, Solo se permitirá un valor de la lista.
--	--

SET('valor1', 'valor2', 'valor3',...)	Puede haber hasta 65 valores, Solo se permitirá un valor de la lista.
---------------------------------------	---

Ejemplo: al definir un campo como ENUM ('A', 'B', 'C'), estamos obligando a que ese campo solo puede tener los valores: "A" o "B" o "C" o NULL.

Valores fuera de rango.

Al trabajar con una base de datos puede ocurrir que al realizar algún cálculo el resultado esté fuera del rango del tipo de dato definido en el campo donde se almacenará. Ante esta situación pueden ocurrir dos cosas: que nos de error y no se almacene; o que se almacene el valor del tipo de dato

más cercano al resultado del cálculo. Pero nunca se almacenará el resultado pues está fuera del rango del tipo de dato.

MySQL reaccionará según el valor de la variable de entorno `sql_mode`:

`SET sql_mode = 'TRADITIONAL';` # Nos dará error y no almacenará nada.

`SET sql_mode = '';` # Avisará de un valor fuera de rango y almacena el valor más cercano.

Supongamos un campo definido como `TINYINT UNSIGNED`, cuyo rango es `[0, 255]` y se intenta almacenar en el valor 400.

```
mysql> create table prueba(numero tinyint unsigned);
Query OK, 0 rows affected (0.01 sec)
mysql> set sql_mode='TRADITIONAL';
Query OK, 0 rows affected (0.00 sec)
mysql> insert into prueba values (400);
ERROR 1264 (22003): Out of range value for column 'numero' at row 1
mysql> select * from prueba;
Empty set (0.00 sec)

mysql> set sql_mode='';
Query OK, 0 rows affected, 1 warning (0.00 sec)
mysql> insert into prueba values (400);
Query OK, 1 row affected, 1 warning (0.00 sec)
mysql> select * from prueba;
+-----+
| numero |
+-----+
|    255 |
+-----+
1 row in set (0.00 sec)

mysql>
```

La variable “`sql_mode`” se puede configurar a nivel de sesión o a nivel global. A nivel de sesión solo le afecta al usuario que la configura y durante su sesión. A nivel global se ha de cambiar con permiso de root y le afecta a todos los usuarios. El formato sería respectivamente:

- `SET GLOBAL sql_mode = 'modo';`
- `SET SESSION sql_mode = 'modo';`

Donde “modo” puede ser:

- **ANSI:** Comprueba la compatibilidad de los datos y si son mayores trunca o pone el valor más cercano. Genera un aviso.
- **STRICT_TRANS_TABLES:** Comprueba la compatibilidad y si no hay compatibilidad aborta la acción. Genera mensaje de error. Por defecto a partir de MySQL 5.7.5
- **TRADITIONAL:** Comprueba la compatibilidad y si no hay aborta la acción. Genera mensaje de error.

La variable `sql_mode` se puede consultar:

```
SELECT @@GLOBAL.sql_mode;
SELECT @@SESSION.sql_mode;
```

Ejercicios.

1. Sobre un servidor MySQL en el que seas administrador realiza las siguientes operaciones:
 - a) Crea tres usuarios, cuyos nombres sean personajes históricos.
 - b) Crea dos bases de datos, cuyos nombres sean barrios de tu localidad de origen.
 - c) Asigna cada una de las bases de datos creadas a dos de los personajes (una a cada uno) y al tercero asigna las dos bases de datos.
 - d) Comprueba que están creadas las bases de datos y los usuarios.
 - e) Sal de MySQL y entra con cada usuario y comprueba que puede ver solo las bases de datos que les has asignado.
 - f) Elimina un usuario y una base de datos.
 - g) Comprueba que se han realizado las eliminaciones.
2. En el servidor de la clase crea una tabla que se llame “mascota” con los siguientes campos:
 - a) Código: numérico y campo clave.
 - b) Nombre: carácter de hasta 20 letras.
 - c) Raza: carácter de 15 letras
 - d) Fecha nacimiento: fecha.
 - e) Nombre del dueño: carácter de hasta 25 letras.

Realiza los siguientes trabajos:

- Añade al menos 5 registros.
 - Consulta todas las mascotas y todos los campos.
 - Consulta solo nombre de mascota y nombre del dueño.
 - Consulta solo nombre de mascota, fecha de nacimiento y nombre de dueño.
3. Modifica el nombre del dueño y fecha de nacimiento de alguna mascota.
 4. Cambia el nombre de la raza de tres mascotas a la vez.
 5. Cambia el nombre de un dueño a “Juan Antonio Rodríguez Fernández”. Si da error modifica el diseño de la tabla ampliando el nombre del dueño a 45 caracteres y vuelve a intentarlo.
 6. Crea una tabla para los dueños de las mascotas, esta tabla se llamará: “amo” y tendrá las siguientes columnas:
 - a) Código_amo: numérico, clave principal y auto incremental.
 - b) Nombre: carácter de 45 letras.
 - c) Dirección: carácter de 50 letras.
 - d) Tfno: carácter de 10 letras.
 7. Toma nota de los códigos de los dueños y modifica la columna de las mascotas para que albergue los códigos de los dueños en lugar de los nombres y actualiza de forma que sea coherente.
 8. Modifica el campo código de dueño de las mascotas para que sea clave foránea. De forma que si se cambia el código del dueño se actualice automáticamente en las mascotas y que no permita eliminar un dueño si tiene mascota.

9. Añade nuevas mascotas con todos los datos. Prueba códigos de dueño que existan y que no existan y analiza los resultados.
10. Cambia el código de algún dueño y comprueba si se ha actualizado en las mascotas.
11. Intenta borrar algún dueño que tenga mascota. ¿Qué ocurre? ¿por qué?
12. Haz las siguientes tareas:
 - a) Cópia de seguridad de las tablas: "mascota" y "amo" que están en el servidor de la clase.
 - b) Edita la copia realizada añadiendo las instrucciones necesarias para que se cree la base de datos: "Veterinariodb" dentro de la cual se crearán las tablas "mascota" y "amo".
 - c) Limpia la copia de seguridad de forma que solo queden los comandos de eliminación y creación de base de datos y tablas, así como los comandos de insertar los registros.
 - d) Instala la copia de seguridad en tu equipo y comprueba que funciona correctamente.